BabyAGI Inside: Compiling a Bounded Agent Loop into Qwen Weights

Yohei Nakajima · Untapped Capital · Preprint, November 2026

Abstract. We demonstrate that a bounded agent loop can be compiled directly into the weights of a pretrained language model, without training. BabyAGI Inside adds a constructed controller, task-ledger protocol, in-model filesystem and halt condition to Qwen3.5-2B. The weights implement the loop; the ledger and files it writes live in inference state. After a single objective prefill, the host only samples tokens until a compiled HALT, with no scheduler, parser, tool executor or logits processor. Recent work compiles machines into Qwen that Qwen calls as tools. Here the compiled loop calls Qwen.

Across ten live traces of two to eight cycles, the checkpoint completed every phase, file operation and halt, and every forced token matched its reference (1,716 of 1,716 positions in the circuit oracle). With the loop inactive, the merged model closely matched stock Qwen on the tested prefixes (top-1 4/4, KL ≤ 0.002). Every emitted token has an owner: in a five-cycle trace, Qwen wrote 1,428 of 3,448 tokens, 1,280 of them reasoning.

Inside the loop, Qwen solved all eight sequential-arithmetic chains run live, up to eight cycles. Structured tasks failed in a revealing way: Qwen wrote a wrong plan and the loop executed it faithfully. The prototype establishes that agent orchestration can run inside a pretrained model's weights, and it separates what the compiled loop does from what the model contributes.

1 Introduction

An LLM agent is usually two programs: a language model, and host code that loops around it. In BabyAGI (2023), a short Python script kept a task queue, called GPT-4 to execute the next task, called it again to create new tasks from the result, called it a third time to reprioritize, and stored results in a vector database. The model only ever produced text; the loop, the queue and the memory lived outside it.

Recent work shows that machines can be compiled directly into a pretrained model's weights. Qwenputer places a RISC-V CPU inside Qwen3.5-4B, using its Gated DeltaNet state as RAM; qwenFS, qwenGPU, qwenTPU and linuxformer add a filesystem, a GPU, a matrix-multiply array and a Linux-booting CPU. In these systems the model remains the agent: Qwen decides when to call the compiled machine, as it would call a tool.

We ask the reverse question. Can the agent loop itself, the part that is usually host code, be compiled into the weights, with the pretrained model as its subroutine? If so, the host shrinks to a sampler, and the compiled controller schedules the workflow phases and delegates designated generation spans to the model, with no prompts or Python deciding what happens next.

Our contributions:

1. **BabyAGI Inside**, a training-free construction that compiles a bounded BabyAGI-style loop into Qwen3.5-2B: a model-written JSON task ledger, a compiled task pointer, bounded model-owned reasoning, qwenFS persistence and a compiled HALT. After one prefill, the host only samples.

2. **A demonstration and verification method** for compiled agents: forced-token circuit oracles, parity checks, exact emission-ownership accounting, and live traces of up to eight cycles that complete and halt.
3. **A capability account** of what the pretrained model does inside the loop: reliable sequential arithmetic, a reasoning-budget curve, and structured-task failures in which the loop carries out a wrong plan faithfully.
4. **A typed in-weight choice gate**, a secondary primitive: one residual coordinate restricts a model-owned decision to legal tokens with no logits processor.

2 Related work

Compiling programs into transformers. RASP and Tracr [2] compile human-readable programs into transformer weights, mainly as ground truth for interpretability; ALTA [3] adds loops and universal transformers. Looped transformers [4] show a fixed, hand-set network can emulate program counters, branches and an instruction-set computer. Percepta compiled a WebAssembly interpreter into a small purpose-built transformer [14]. These models are built from scratch; none hosts a pretrained language model. Tracr-injection [5] distills compiled algorithms into a pretrained LLM, but by fine-tuning.

Machines compiled into a pretrained Qwen. Our closest neighbours compile machines into Qwen3.5 without training and leave Qwen's own weights unchanged. Qwenputer [6] adds a RISC-V CPU and an in-model filesystem, using Gated DeltaNet state as exact RAM; qwenFS [7], on which we build, adds a filesystem to Qwen3.5-2B; qwenGPU [8] a SIMT GPU that runs CUDA; qwenTPU [9] a matrix-multiply array; and linuxformer [10] a CPU that boots Linux, with Qwen using the shell. In these systems Qwen is the controller and the machine is its tool. The nearest exception is Agrawal's compiled oversight monitor [18], which inserts a flag into Qwen's output and logs it whatever Qwen is told; it interrupts the model rather than running a task loop around it. qwenFS itself already parses tool calls and frames responses in-weight; we lift that control to the level of a whole task workflow.

Internalizing workflows and compiled modules. Dennis et al. [15] compile agentic workflows into a small model's weights by generating procedural conversations and fine-tuning, removing the external orchestrator. Neural Comprehension [16] gates compiled neural modules with a pretrained language model, token by token, for symbolic operations. Our construction is training-free: the controller is specified analytically and schedules repeated generation and storage operations across a multi-cycle task loop.

Program-controlled LLMs. ReAct-style [11] agent loops let the model decide control flow. LMQL [17] constrains generation inside an explicit program. LLM-as-Code [12] argues against model-owned control: a program should own looping and sequencing, and call the LLM only for judgment. BabyAGI Inside adopts that division of labour, but moves the program from the host into the weights.

Constrained decoding and test-time budgets. Grammar-constrained decoding masks illegal tokens in a host-side logits processor. Our typed gate implements the same restriction inside the network. Budget forcing [13] controls test-time reasoning by ending or extending thinking at a token count; our controller compiles a fixed budget into each execute phase.

3 System

BabyAGI Inside keeps the division of labour of the original script: what Python did is compiled; what GPT-4 did, Qwen does. The workflow is simplified: one up-front plan, fixed-order prioritization, and a cycle count fixed at build.

3.1 Base model and layout

We start from Qwen3.5-2B and the qwenFS merge, which prepends compiled filesystem layers to Qwen's 24 and widens the residual stream. Our controller adds its own registers and a prompt ROM. The merged stream is 4,096 wide: Qwen's native dimensions 0–2047, qwenFS 2048–3071, agent scratch 3072–3208, and padding. Qwen's weights are unchanged apart from the norm-gain rescaling that the widening requires. No parameter is trained. The weights implement the controller and filesystem; the ledger and files a run produces live in its inference state (attention history and DeltaNet state) and persist across cycles within that run.

3.2 The loop

After the objective prefill and a START symbol, the controller steps through compiled phases. In the current protocol (ledger-pointer):

1. **Create.** The ROM prints the creation prompt and opens a `write_file` call; Qwen writes a JSON task ledger. qwenFS stores it.
2. **Point.** A compiled pointer advances to the next ledger entry. Prioritization is fixed-order and owned by the circuit.
3. **Execute.** The ROM opens a bounded reasoning span of 256 tokens, which Qwen fills, then forces the span closed and opens a write; Qwen writes the result.
4. **Persist.** qwenFS stores `result.txt`; the next cycle reads it back token-exactly.
5. **Halt.** After N cycles the controller emits HALT.

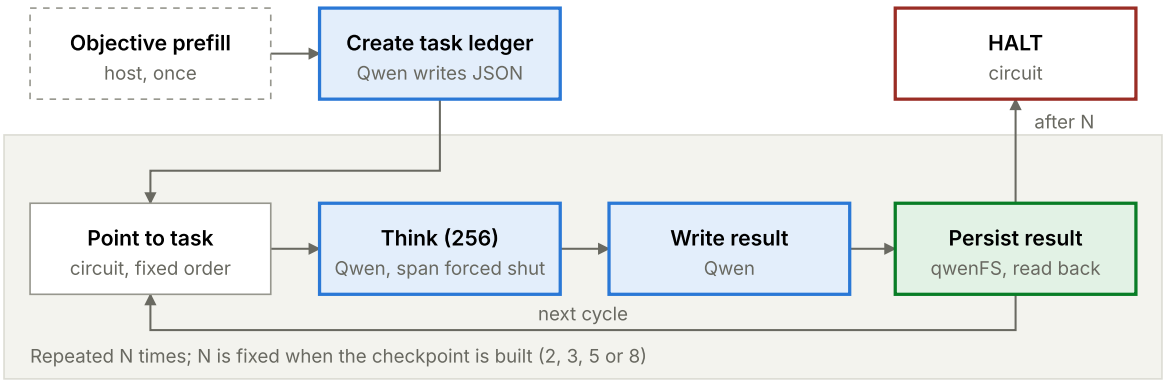


Figure 1. The ledger-pointer protocol: one cycle, repeated N times. Every transition between boxes is compiled; Qwen writes only inside the blue boxes. Green is the compiled filesystem (qwenFS); gray and red are compiled controller phases.

N is fixed when the checkpoint is built; “bounded” refers to this cycle count. We build N = 2, 3, 5 and 8; an eight-cycle run comprises 24 phases.

3.3 A primitive: the typed choice gate

An earlier protocol (typed, v17) let Qwen prioritize with a single legal token. The ROM opens `read_file(task`, Qwen supplies one token, and the ROM closes `.txt)`, so the choice becomes part of a filename. One added residual coordinate is nonzero only for tokens `1` and `2`. At the decision position, a compiled layer activates it and the final norm amplifies it, adding the same large value to both legal logits. Every illegal token loses by at least 1,893 logits; how the gap between `1` and `2` fares under bf16 rounding at this offset is untested. The current protocol uses the compiled pointer instead, after two diagnostic probes in which Qwen chose Task 1 in both orders (Section 6.5).

3.4 The host

The host prefills one objective, then calls `sample()` until HALT. It holds no scheduler, parser, tool executor, retry logic or logits processor. Every prompt, tool call, file response and control token after the objective is emitted by the checkpoint itself. The complete post-prefill runtime:

```
ids = prefill(objective) + [START]
while ids[-1] != HALT:
    ids.append(argmax(model(ids)))   # KV-cached in practice
```

4 Verification methodology

A compiled agent makes two kinds of claim, and we test them separately: that the circuit did exactly what it was built to do, and that Qwen did something useful inside it.

Circuit oracle. We script a full trace, feed it token by token, and require the checkpoint to predict every compiled token exactly: prompts, tool-call wrappers, filesystem responses and control symbols. A separate check confirms that every model-owned reasoning position yields an ordinary vocabulary token.

Parity. With the loop inactive, the merged checkpoint must match stock Qwen3.5-2B: we compare top-1 predictions and KL divergence on native text.

Token-ownership partition. For every live trace, each emitted token is attributed to exactly one owner: ROM prompt, filesystem response, controller symbol, or model. Model tokens are further split into reasoning, payload and tool-call closure. The partition must account for every token.

Host-versus-compiled equivalence. We run the same protocol with stock Qwen and a Python host, and compare model-owned outputs. Exact equivalence is only expected at the output level: in the reasoning spans, small numerical differences in the widened model accumulate over 256 greedy tokens.

Host proxy, and its limits. A host-side harness with stock weights is fast enough to screen prompts and run paired baselines. It is approximate. The compiled control symbol has a nonzero view in Qwen's native dimensions (0.25 × the newline embedding), which the first harness omitted; qwenFS responses use extended embeddings that stock Qwen cannot consume, so the harness feeds ordinary-token surrogates. Live compiled traces are therefore authoritative for any claim about compiled behaviour.

Scope of the claims. Our validation is empirical and limited to the tested traces, checkpoints and backend (Apple MPS, bf16). A correctness contract over arbitrary model outputs, including a total token bound for every model-owned span, is future work.

Preregistration. Benchmarks, prompts, decoding (greedy), token budgets, scorers and trace-selection rules are frozen and checksummed before each run. Superseded results are kept and labelled (Appendix A).

5 Results I: the loop runs as built

Every systems check we ran passed, on every checkpoint and every live trace.

- **Circuit oracle:** 1,716/1,716 forced tokens exact on the 256-token-budget checkpoint; 1,280/1,280 model-owned reasoning positions yield legal vocabulary tokens. Earlier checkpoints passed 4,788/4,788 (typed, v17) and 3,819–3,988 (five-cycle variants).
- **Parity, loop inactive:** top-1 agreement 4/4, maximum KL 0.0019 against stock Qwen3.5-2B.
- **Typed gate:** at all five decision positions of a v17 trace, the best illegal token trails the legal pair by at least 1,893 logits.
- **Host versus compiled:** model-owned ledger and result outputs matched in 6/6 stages; reasoning spans matched token-for-token in 0/5.

5.1 Live compiled traces

All ten live traces completed every phase, file operation and HALT, at up to eight cycles (Table 1).

Table 1. Live compiled traces on the ledger-pointer checkpoints (greedy, bf16, Apple M5).

Item	Cycles	Controller, files, HALT	Benchmark	Result sequence
arithmetic-add-2	2	pass	pass	9, 18
arithmetic-add-3	3	pass	pass	11, 22, 19
arithmetic-add-5	5	pass	pass	15, 30, 27, 32, 96
arithmetic-add-8	8	pass	pass	21, 42, 39, 44, 132, 124, 248, 257
arithmetic-multiply-2	2	pass	pass	21, 25
arithmetic-multiply-3	3	pass	pass	24, 29, 25
arithmetic-multiply-5	5	pass	pass	30, 37, 33, 66, 73
arithmetic-multiply-8	8	pass	pass	39, 49, 45, 90, 97, 194, 185, 196
structured-even-8	8	pass	fail	wrong ledger; 0/8 transitions
structured-odd-8	8	pass	fail	wrong ledger; 0/8 transitions

The four add-first traces were chosen by a preregistered rule; the other six were chosen afterwards for diagnosis. They are evidence of what the compiled loop does, not an accuracy estimate.

5.2 Who wrote the tokens

Emission ownership is accounted for exactly in all ten traces: every token has one owner. Ownership records who selected each emitted token, not how much of the run's competence each part supplies. In the five-cycle

multiply-first trace, Qwen wrote 1,428 of 3,448 tokens (41%), including 1,280 reasoning tokens; the other 2,020 were forced by the ROM, qwenFS or the controller. Without reasoning spans the picture is very different: the v17 typed trace emitted 5,010 tokens, of which Qwen wrote 241 (4.8%), and its five prioritization decisions were one token each.

5.3 Cost

On an Apple M5 (MPS, bf16), the v17 trace ran at about 10.4 tokens per second (5,010 tokens in 481 s). Building a 2B checkpoint takes about 32 s and peaks near 14.6 GB of memory. The compiled loop pays one decode step for every forced token, where a host loop could prefill known prompt text in a single pass; packaging the workflow in the checkpoint costs throughput.

6 Results II: capability is narrow

Qwen solved all eight arithmetic chains tested live; both structured-task plans it wrote were wrong. Whether the loop helps over plain chat is not yet established.

6.1 Reasoning budget (exploratory)

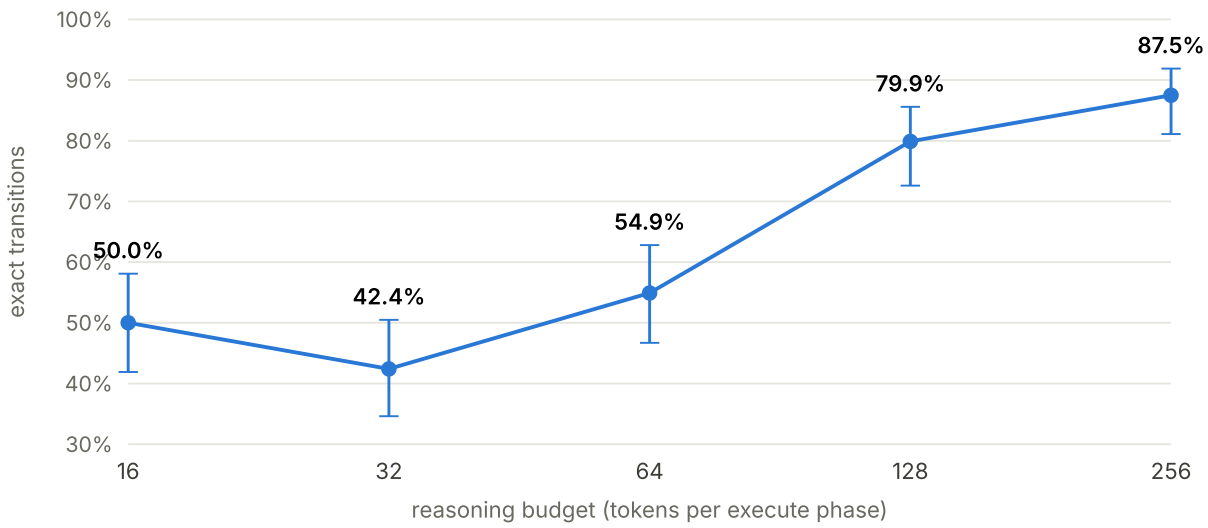


Figure 2. Transition accuracy by reasoning budget (exploratory): 144 transitions per budget from the frozen v3 suite, Qwen3.5-2B, greedy, bf16 on MPS. Whiskers are 95% intervals.

More reasoning substantially lifted transition accuracy under the tested protocol. Each transition starts from the correct task and incoming state, so the curve measures conditional execution, not planning. Gains are largest between 64 and 128 tokens; 128 to 256 is a modest further gain (17 improvements, 6 regressions; paired $p = 0.035$, but the objective-clustered interval touches zero). The curve is not monotonic: a fixed span changes the model's trajectory, so more tokens can make an answer worse. At 256 tokens, per-step accuracy is 91.7% on ordered literals, 94.4% on arithmetic, 86.1% on structured operations and 77.8% on constrained language edits. We froze 256 as the protocol budget.

6.2 Whole chains

Per-step accuracy does not carry over to whole chains. On the 32-item suite, plain Qwen solved 5 chains in one turn; a host-side proxy of the loop solved 7 (Table 2). A live compiled run of the full suite is under way.

Table 2. Whole-chain accuracy on the frozen 32-item suite (144 transitions).

Arm	Exact chains	Exact transitions
Plain Qwen, one turn	5/32 (15.6%)	23/144 (16.0%)
Host proxy of the loop	7/32 (21.9%)	35/144 (24.3%)

The proxy had three improvements and one regression against plain chat (exact McNemar $p = 0.625$; item bootstrap interval −6.25 to +18.75 points). That does not establish an improvement. All seven proxy successes are sequential arithmetic; ordered-literal, structured and language chains are 0/8 in both arms. A first run reported 9/32 for the proxy; it is withdrawn (Appendix A).

6.3 What the live traces show

The compiled checkpoint solved all eight arithmetic chains we ran live, including eight-cycle chains with mixed operations. In one case it did better than the proxy predicted: on the five-cycle multiply-first chain the proxy stalled while closing its third write, while the live checkpoint produced 30, 37, 33, 66, 73 exactly. Proxy results are screening evidence; live traces decide.

6.4 The loop executes a wrong plan perfectly

Both eight-cycle structured traces completed all 24 phases and halted, with exact file operations, and scored 0/8. The fault is in the plan Qwen wrote: its ledger's first task added the combined amount and every later task added one. The controller then executed that plan faithfully for eight cycles. The task errors arose in the plan Qwen wrote, while the control protocol executed as built.

Plan quality is also hard to score. All 32 proxy ledgers parsed and 31/32 extracted the correct initial value, but exact task-string match was 0/32, because close paraphrases such as “multiply by 2” do not equal “multiply the result by 2”. We keep that frozen metric and will report lexical and semantic plan fidelity separately in future suites.

6.5 What did not work

- **No reasoning span:** the typed v17 protocol, with 2–8-token payloads and no thinking, solved 0/12 held-out items.
- **Few-shot exemplars:** format-only examples fixed first-task shape but leaked content; a “launch note” exemplar became an attractor for a launch-announcement objective.
- **A model-maintained ledger lifecycle:** when Qwen rewrote the whole ledger each phase, it left completed tasks pending and regenerated candidate lists instead of executing.
- **Model-owned prioritization:** with the typed gate, Qwen chose Task 1 in both orders of a counterbalanced probe (n = 2), a positional bias. The current protocol uses a compiled pointer instead.

7 Discussion and limitations

What compiling the loop buys. The workflow ships inside the checkpoint: loading the model and sampling is the whole runtime. Control flow is fixed by construction rather than by prompts or host code, and on every tested trace the loop ran its phases in order, persisted each result and halted. Every emitted token has an owner, so the model's share of a run is measurable (41% in our five-cycle trace, 4.8% without reasoning spans), and failures localize: the structured traces show a wrong plan carried out by a loop that ran as built.

What it does not buy. This study shows no capability gain from compilation. Exact orchestration around a 2B model still yields narrow capability: arithmetic chains succeeded and the two tested structured-task plans were wrong, and we have not shown the loop beats plain chat.

Limitations.

- **Bounded loop.** N is fixed at build time; each length needs its own checkpoint, and the controller does not decide when to stop.
- **Compiled prioritization.** The current protocol's task pointer is fixed-order. BabyAGI's prioritization agent is represented only in the earlier typed protocol, where two probes suggested a positional bias.
- **Up-front planning.** Qwen writes the whole ledger at the start. The original BabyAGI created tasks incrementally from each result, which may matter most for the structured failures.
- **Output agreement only.** Outputs agreed in the six stages tested; reasoning traces differed.
- **Approximate proxy.** Host-side baselines cannot consume qwenFS's extended embeddings exactly.
- **Small samples.** 32 chains cannot detect modest effects, and our ten live traces were not a representative sample. Edge cases are untested: we have not exercised empty, malformed or non-closing model outputs.
- **One model, one machine.** All results are for Qwen3.5-2B on an Apple M5 (MPS, bf16). Related projects found bf16 bugs that appeared only on particular GPUs; we have not yet replicated on CUDA.

Future work. A program-counter controller held in Gated DeltaNet state, as in Qwenputer and linuxformer, would remove the cycle bound and allow model-decided halting through the typed gate. Incremental task creation and a larger, new frozen suite would test whether the loop can help planning. Fine-tuning only Qwen's dimensions, with the compiled parts frozen, would test how much of the gap is the pretrained model's. Giving both loops a correct reference plan and the same model-written plan would measure the planning bottleneck directly.

8 Conclusion

An agent loop can be compiled into a pretrained model's weights without training. In BabyAGI Inside, the controller and filesystem live in Qwen3.5-2B's weights, the ledger and files in its inference state, and the host only samples. On every tested trace, up to eight cycles, the loop ran as built and halted. Inside it, Qwen solved arithmetic chains and wrote wrong plans for structured tasks, which the loop then carried out faithfully. Fixing orchestration by construction makes the model's contribution, and its failures, directly visible.

Appendix

A. Superseded results

- **Host loop 9/32 (withdrawn).** The first paired run omitted the compiled control symbol's view in Qwen's native dimensions (0.25 × the newline embedding). Corrected, the proxy scores 7/32; its two structured successes were false positives. The 9/32 run is kept as an audit artifact (commit `a5fbf2f`; corrected in `14f1c0f`).
- **Held-out v1, 0/12 (typed protocol, no reasoning span).** Superseded as a capability measure by the budget study and the v3 suite.
- **Pilot budget curve (32 transitions).** A sampling bug was found before cross-budget comparison; those outputs were excluded and the full 144-transition run replaces them.

B. Reproduction

Checkpoints: `build/babyagi-2b-ledger-pointer-b256-m{2,3,5,8}`. Results: `results/end-to-end-v3-b256/` (paired run and ten live traces), `results/end-to-end-v3-b256-control-surrogate/` (corrected proxy), `results/execution-budget-v3-full/` (budget curve). Frozen v3 suite SHA-256 `e880e043…dff4`.

```
uv run python -m qfs.end_to_end models/Qwen3.5-2B \
  benchmarks/babyagi-v3.json \
  results/end-to-end-v3-b256-control-surrogate \
  --think-tokens 256 --plain-answer-tokens 256 \
  --control-surrogate-scale 0.25 \
  --plain-from results/end-to-end-v3-b256 --device mps
```

Build, circuit-oracle and live-decode commands for the ledger-pointer checkpoints are provided in the code repository; the five-cycle multiply-first trace should reproduce the result sequence 30, 37, 33, 66, 73.

References

1. Nakajima, Y. BabyAGI (2023). github.com/yoheinakajima/babyagi
2. Lindner et al. Tracr: Compiled Transformers as a Laboratory for Interpretability. NeurIPS 2023. arXiv:2301.05062
3. Shaw et al. ALTA: Compiler-Based Analysis of Transformers. TMLR 2025. arXiv:2410.18077
4. Giannou et al. Looped Transformers as Programmable Computers. ICML 2023.
5. Tracr-Injection: Distilling Algorithms into Pre-trained Language Models. arXiv:2505.10719
6. Agrawal, D. Qwenputer (2026). github.com/divagr18/qwenputer
7. Baraiya, V. qwenFS (2026). github.com/vixhal-baraiya/qwenFS
8. Baraiya, V. qwenGPU (2026). github.com/vixhal-baraiya/qwenGPU
9. qwenTPU (2026). github.com/Bobliuuu/qwentpu
10. linuxformer (2026). github.com/itsrealranky/linuxformer
11. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629
12. LLM-as-Code: Agentic Programming for Agent Harness (2026). arXiv:2606.15874

13. Muennighoff et al. s1: Simple test-time scaling (2025). arXiv:2501.19393
14. Percepta. Can LLMs Be Computers? (2026). Blog post.
15. Dennis, S., Patil, R., Shabahang, K., Guo, H. Compiling Agentic Workflows into LLM Weights (2026). arXiv:2605.22502
16. Weng et al. Mastering Symbolic Operations: Augmenting Language Models with Compiled Neural Networks. arXiv:2304.01665
17. Beurer-Kellner et al. Prompting Is Programming: A Query Language for Large Language Models (LMQL). PLDI 2023. arXiv:2212.06094
18. Agrawal, D. Building oversight tools into a model's weights (2026). divagr.com